

- ❗ Естественно, в игре должно быть какое-никакое звуковое сопровождение. Звук добавляет ощущение взаимодействия - звуки наведения на кнопки, нажатия, покупка юнитов, улучшение башни. Фоновая музыка может задавать настроение.

SoundSO.

Создаём конфиг ScriptableObject для звука. Он будет хранить название, сам звуковой клип, громкость клипа (должна быть относительно выровнена), зацикленность и тип звука.

SoundType представляет все типы звуков - Music, SFX, LoopSFX, UI. Позже каждый из них будет обрабатываться по разному.

SoundEnum был сделан для того, чтобы были универсальные звуки, дабы не прокидывать во все кнопки один и тот же SoundSO. При наведении будем просто вызывать PlaySFX(SoundEnum).

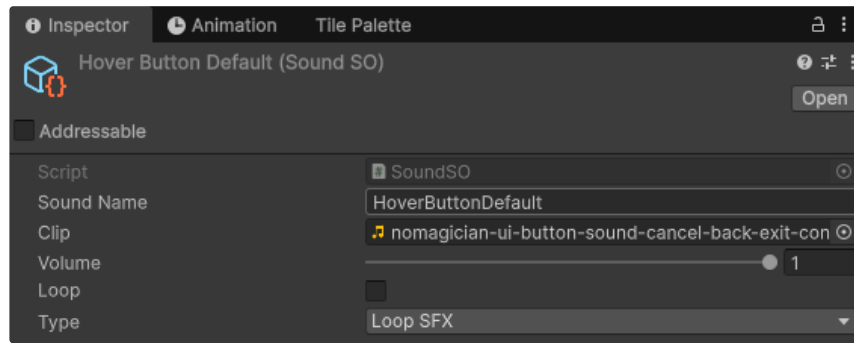
MusicType - тип принадлежности фоновой музыки. Для боевых сцен может быть массив разных аудио. При запуске уровня музыка будет выбираться случайно из доступных.

```
1 using UnityEngine;
2
3 [CreateAssetMenu(menuName = "Audio/Sound")]
4 public class SoundSO : ScriptableObject
5 {
6     public string soundName;
7     public AudioClip clip;
8     [Range(0, 1)] public float volume = 1f;
9     public bool loop = false;
10    public SoundType type = SoundType.SFX;
11 }
12
13 public enum SoundType
14 {
15     Music,
16     SFX,
17     LoopSFX,
18     UI
19 }
20
21 public enum SoundEnum
22 {
23     ClickButtonDefault,
24     HoverButtonDefault
25 }
26
27 public enum MusicType
28 {
29     Menu,
```

```

30     FightRandom,
31     Boss
32 }

```



SoundSO.

AudioSourcePool.

- Учитывая, что работаем мы с SO. Нам всё таки придётся создавать AudioSource типы и записывать в них конфиг из SO. Чтобы избежать множества аллокаций, частых сборок мусора, мы сделаем отдельный пул для аудио сурс. Логика этого пула отличается от обычного GameObject, который мы использовали для врагов, поэтому он будет создан отдельно.

```

1 using System.Collections.Generic;
2 using UnityEngine;
3
4 public class AudioSourcePool : MonoBehaviour
5 {
6     private Queue pool = new Queue();
7     private Transform parent;
8     private int expandSize;
9     private GameObject sourcePrefab;
10
11     private HashSet activeSources = new HashSet
12     public IReadOnlyCollection ActiveSources => activeSour
13
14     public AudioSourcePool(GameObject prefab, int initialSize, int expa
15     {
16         this.parent = parent;
17         this.expandSize = Mathf.Max(10, expandSize);
18         sourcePrefab = prefab;
19         CreateBatch(initialSize);
20     }
21
22     private void CreateBatch(int count)
23     {
24         for (int i = 0; i < count; i++)
25         {
26             GameObject go = GameObject.Instantiate(sourcePrefab, parent
27             go.SetActive(false);
28             pool.Enqueue(go.GetComponent());
29         }
30     }
31
32     public AudioSource Get()
33     {
34         if (pool.Count == 0) CreateBatch(expandSize);

```

```

35
36     AudioSource source = pool.Dequeue();
37     source.gameObject.SetActive(true);
38
39     activeSources.Add(source);
40
41     return source;
42 }
43
44 public void Return(AudioSource source)
45 {
46     if (!activeSources.Remove(source))
47         return;
48
49     source.Stop();
50     source.clip = null;
51     source.pitch = 1f;
52
53     source.gameObject.SetActive(false);
54
55     pool.Enqueue(source);
56 }
57 }

```

AudioManager.

```

1 using NUnit.Framework;
2 using System.Collections;
3 using System.Collections.Generic;
4 using Unity.VisualScripting;
5 using UnityEditor.AddressableAssets.Build;
6 using UnityEngine;
7
8
9 public class AudioManager : Manager<AudioManager>
10 {
11     [Header("AudioSource Prefab")]
12     public GameObject audioSourcePrefab;
13
14     [Header("Pool Settings")]
15     public int initialPoolSize = 10;
16     public int expandPoolSize = 10;
17
18     private AudioSourcePool sfxPool;
19     private readonly HashSet<AudioSource> loopSources =
20         new HashSet<AudioSource>();
21
22     [Header("Music Sources for crossfade")]
23     private AudioSource musicSourceA;
24     private AudioSource musicSourceB;
25     private bool usingSourceA = true;
26
27     private float musicVolume = 1f;
28     private float sfxVolume = 1f;
29     private Coroutine musicCoroutine;
30     private float currentPitch = 1f;
31
32     [Header("UISource")]
33     private AudioSource uiSource;
34
35     [Header("AudioLibrary")]
36     [SerializeField] private AudioLibrary library;
37
38
39     protected override void Awake()

```

```

40     {
41         base.Awake();
42
43         sfxPool = new AudioSourcePool(audioSourcePrefab, initialPoolSiz
44
45         musicSourceA = new GameObject("MusicSourceA").AddComponent<Audi
46         musicSourceB = new GameObject("MusicSourceB").AddComponent<Audi
47         musicSourceA.transform.SetParent(transform);
48         musicSourceB.transform.SetParent(transform);
49
50         musicSourceA.loop = true;
51         musicSourceB.loop = true;
52
53         uiSource = new GameObject("UISource").AddComponent<AudioSource>
54         uiSource.transform.SetParent(transform);
55         uiSource.loop = false;
56         uiSource.ignoreListenerPause = true;
57
58         library.Init();
59     }
60
61     private void Start()
62     {
63         LoadSettings();
64
65         SettingsService.Instance.OnMusicVolumeChanged += SetMusicVolume
66         SettingsService.Instance.OnSFXVolumeChanged += SetSFXVolume;
67
68         SpeedGameManager.Instance.OnSpeedGameChanged += HandleSpeedChan
69     }
70
71     private void LoadSettings()
72     {
73         musicVolume = SettingsService.Instance.MusicVolume;
74         sfxVolume = SettingsService.Instance.SFXVolume;
75
76         musicSourceA.volume = musicVolume;
77         musicSourceB.volume = musicVolume;
78     }
79

```

PlaySFX(SoundSO) - одновременно производит звук типа SFX, а также запускает корутину, которая по завершению звука вернёт компонент в пул.

PlayUISFX(SoundSO) - одновременно производит звук UI типа. Отличие его в том, что он играет одинаково вне зависимости от скорости игры. Более того, пауза не прерывает его проигрывание.

PlayLoopSFX(SoundSO) - начинает производство зацикленного звука (лазерная башня, к примеру). Их мы храним в отдельной коллекции, чтобы можно было в случае чего поставить на паузу и возобновить (когда игрок нажмёт паузу).

```

1     public void PlaySFX(SoundSO sound)
2     {
3         if (sound.type != SoundType.SFX)
4             return;
5
6         AudioSource source = sfxPool.Get();

```

```

7      source.clip = sound.clip;
8      source.volume = sound.volume * sfxVolume;
9      source.loop = false;
10     source.pitch = currentPitch;
11     source.Play();
12
13     StartCoroutine(ReturnToPoolAfterPlay(source));
14 }
15
16 private IEnumerator ReturnToPoolAfterPlay(AudioSource source)
17 {
18     yield return new WaitWhile(() => source.isPlaying);
19     sfxPool.Return(source);
20 }
21
22
23 public void PlayUISFX(SoundSO sound)
24 {
25     if (sound.type != SoundType.UI) return;
26
27     uiSource.pitch = 1f;
28     uiSource.PlayOneShot(sound.clip, sound.volume * sfxVolume);
29 }
30
31 public AudioSource PlayLoopSFX(SoundSO sound)
32 {
33     if (sound.type != SoundType.LoopSFX)
34         return null;
35
36     AudioSource source = sfxPool.Get();
37
38     source.clip = sound.clip;
39     source.volume = sound.volume * sfxVolume;
40     source.loop = true;
41     source.pitch = currentPitch;
42
43     source.Play();
44
45     loopSources.Add(source);
46
47     return source;
48 }

```

StopLoopSFX(AudioSource) - для остановки конкретного зацикленного звука необходимо иметь ссылку на этот компонент. После чего мы сможем вернуть его в пул, а также убрать из нашей коллекции.

PauseLoopSFX() - просто ставит на паузу все зацикленные звуки.

ResumeLoopSFX() - перебирает все звуки и включает их обратно.

```

1 public void StopLoopSFX(AudioSource source)
2 {
3     if (source == null)
4         return;
5
6     loopSources.Remove(source);
7     sfxPool.Return(source);
8 }
9
10 private void PauseLoopSFX()
11 {
12     foreach(var source in loopSources)

```

```

13     {
14         source.Pause();
15     }
16 }
17
18 private void ResumeLoopSFX()
19 {
20     foreach(var source in loopSources)
21     {
22         source.UnPause();
23     }
24 }

```

PlayMusic(SoundSO, float) - запускает проигрывание музыки со звуковым переходом. Мы берём текущую активную музыку, а также ту, что нам предстоит включить. Устанавливаем громкость второй на 0 и начинаем проигрывание. Сразу прокидываем их в корутину CrossFadeMusic.

CrossFadeMusic(AudioSource, AudioSource, float) - за период перехода, относительно deltaTime, через Lerp плавно сбрасывает громкость одной, и прибавляет громкость другой. После чего останавливает первую.

SetMusicVolume(float) - устанавливает громкость текущей активной музыки.

SetSFXVolume - устанавливает громкость для SFX.

```

1
2 public void PlayMusic(SoundSO music, float fadeDuration = 2f)
3 {
4     if (music.type != SoundType.Music) return;
5
6     AudioSource active = usingSourceA ? musicSourceA : musicSourceB
7     AudioSource next = usingSourceA ? musicSourceB : musicSourceA;
8
9     next.clip = music.clip;
10    next.volume = 0;
11    next.loop = music.loop;
12    next.Play();
13
14    musicCoroutine = StartCoroutine(CrossFadeMusic(active, next, fa
15    usingSourceA = !usingSourceA;
16 }
17
18 private IEnumerator CrossFadeMusic(AudioSource from, AudioSource to
19 {
20     float t = 0f;
21     while (t < duration)
22     {
23         t += Time.deltaTime;
24         from.volume = Mathf.Lerp(musicVolume, 0, t / duration);
25         to.volume = Mathf.Lerp(0, musicVolume, t / duration);
26         yield return null;
27     }
28     from.Stop();
29     from.clip = null;
30     to.volume = musicVolume;
31
32     musicCoroutine = null;
33 }
34

```

```

35
36     public void SetMusicVolume(float value)
37     {
38         musicVolume = value;
39         musicSourceA.volume = usingSourceA ? musicVolume : 0;
40         musicSourceB.volume = usingSourceA ? 0 : musicVolume;
41     }
42
43     public void SetSFXVolume(float value) => sfxVolume = value;

```

PlaySFXType(SoundEnum) - запускает звук из библиотеки частых звуков. Например, для кнопок у меня есть два стандартных звука. И мне не придётся указывать на них ссылку в скрипте кнопок, а достаточно будет вызвать этот метод и передать соответствующий тип.

PlayUISFXType(SoundEnum) - проигрывает именно UISFX (как-раз кнопки). Позволяет производить звуки даже на паузе.

PlayMusicType(MusicType) - можно также прокинуть случайную из доступных фоновую музыку по её типу.

```

1     public void PlaySFXType(SoundEnum sound)
2     {
3         var clip = library.GetClip(sound);
4         if (clip != null)
5         {
6             PlaySFX(clip);
7         }
8     }
9
10    public void PlayUISFXType(SoundEnum sound)
11    {
12        var clip = library.GetClip(sound);
13        if (clip != null)
14        {
15            PlayUISFX(clip);
16        }
17    }
18
19    public void PlayMusicType(MusicType type)
20    {
21        var music = library.GetMusic(type);
22        if (music != null)
23        {
24            PlayMusic(music);
25        }
26    }

```

StopMusic(float) - плавно останавливает музыкальное сопровождение.

FadeOutAndStop(AudioSource, float) - как раз таки берёт текущее сопровождение и плавно скручивает его громкость, а позже останавливает.

```

1     public void StopMusic(float fadeDuration = 1.5f)
2     {
3         if (musicCoroutine != null)
4         {
5             StopCoroutine(musicCoroutine);
6             musicCoroutine = null;
7         }

```

```

8
9     if (musicSourceA.isPlaying)
10         StartCoroutine(FadeOutAndStop(musicSourceA, fadeDuration));
11
12     if (musicSourceB.isPlaying)
13         StartCoroutine(FadeOutAndStop(musicSourceB, fadeDuration));
14 }
15
16 private IEnumerator FadeOutAndStop(AudioSource source, float fadeDu
17 {
18     float startVolume = source.volume;
19     float t = 0f;
20
21     while (t < fadeDuration)
22     {
23         t += Time.deltaTime;
24         source.volume = Mathf.Lerp(startVolume, 0, t / fadeDuration);
25         yield return null;
26     }
27
28     source.Stop();
29     source.clip = null;
30     source.volume = musicVolume;
31
32 }

```

Мы подписались на изменение скорости игры. Событие, которое вызывает SpeedGameManager. В этом методе мы вызываем методы ResumeMusic, ResumeLoopSFX, SetPitchBySpeed.

PauseAllGameAudio() - ставит на паузу все существующие звуки. При этом короткие звуки не ставятся на паузу, а выключаются.

PauseMusic() - ставит играющий звук на паузу.

ResumeMusic() - UnPause.

StopAllSFX() - берёт все активные звуки. Сверяет их с массивом зацикленных, и если они совпадают, то пропускает, т.к. для них у нас своя логика.

SetPitchBySpeed(float) - ускоряет скорость аудио, но не соразмерно скорости игры. Если ставить на x4 скорость и делать 4.0 Pitch, то звук будет вообще не разборчив. Поэтому через Lerp мы вычисляем немного иное значение. На x4 скорости питч звуков будет равен 1.5.

```

1     private void HandleSpeedChanged(GameSpeed speed)
2     {
3         if (speed == GameSpeed.Pause)
4         {
5             PauseAllGameAudio();
6             return;
7         }
8
9         ResumeMusic();
10        ResumeLoopSFX();
11        SetPitchBySpeed(SpeedGameManager.Instance.SpeedMultiplier);
12    }

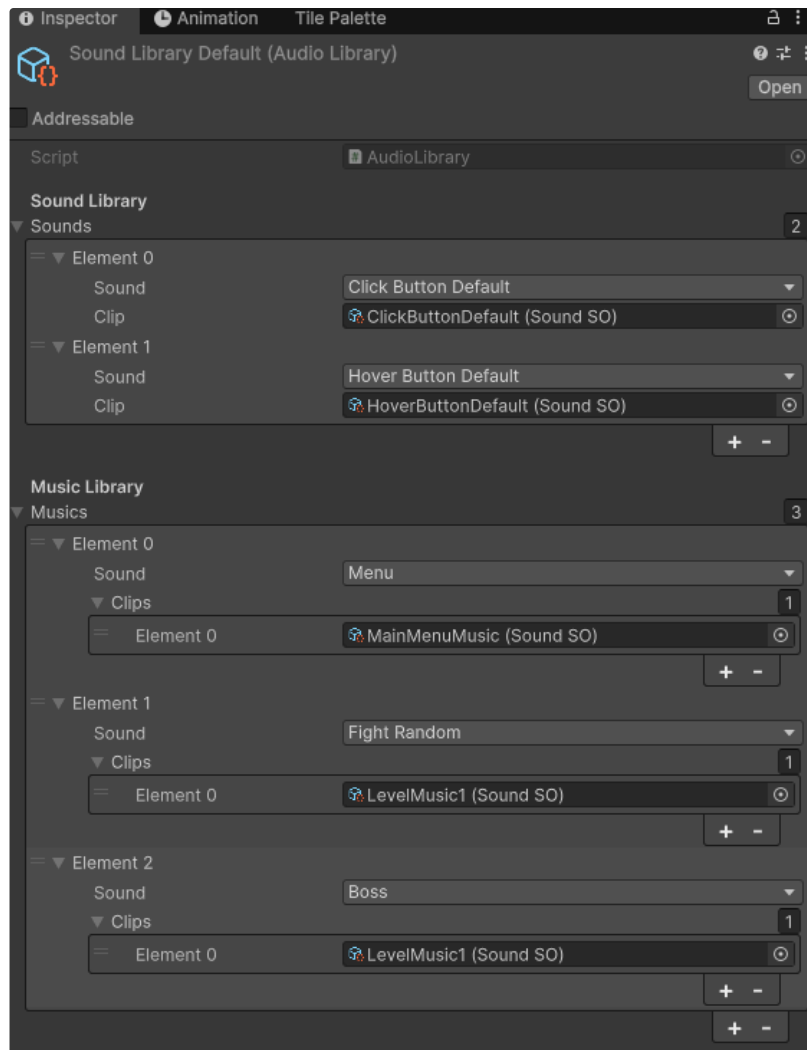
```



```

13
14     private void PauseAllGameAudio()
15     {
16         PauseMusic();
17         StopAllSFX();
18         PauseLoopSFX();
19     }
20
21     public void PauseMusic()
22     {
23         if (musicSourceA.isPlaying) musicSourceA.Pause();
24         if (musicSourceB.isPlaying) musicSourceB.Pause();
25     }
26
27     public void ResumeMusic()
28     {
29         musicSourceA.UnPause();
30         musicSourceB.UnPause();
31     }
32
33     private void StopAllSFX()
34     {
35         var active = new List<AudioSource>(sfxPool.ActiveSources);
36
37         foreach (var source in active)
38         {
39             if (loopSources.Contains(source))
40                 continue;
41
42             source.Stop();
43             sfxPool.Return(source);
44         }
45     }
46
47     private void SetPitchBySpeed(float speed)
48     {
49         if (speed <= 1f)
50             currentPitch = 1f;
51         else
52             currentPitch = Mathf.Lerp(1f, 1.5f, (speed - 1f) / 3); //~
53
54         foreach (var source in sfxPool.ActiveSources)
55         {
56             source.pitch = currentPitch;
57         }
58     }
59 }

```



Библиотека стандартных звуков и библиотека музыкального сопровождения.

Сейчас вызов звуков работает через жесткую связность. Вызывать приходится звук напрямую через `AudioManager.Instance.PlaySFX()`. Для небольшого проекта это нормально. В более крупных проектах используют Ivent-Driven систему (IventBus).

Если враг умирает, то он прокидывает звук в глобальный менеджер. Враг знает про `AudioManager`. В IventDriven враг при смерти вызывает событие, куда передаёт данные. Допустим у нас есть статический класс `AudioEvents` с событием `PlaySFX(AudioSO)`. В таком случае `AudioManager` подписывается на событие и будет проигрывать любой пришедший звук, не зная даже от кого он пришёл. А враг, при смерти будет вызывать `AudioEvents.PlaySFX(stats.deathSound);` И враг ничего не знает о менеджере. У них есть прослойка. Враг не знает методов и способов воспроизведения звука. А менеджер может меняться и просто реагировать на абстрактное событие. К тому же, какой-нибудь отладочный класс для логгирования также сможет просто подписаться на событие.



Аудио менеджер ещё может поменяться, но даже сейчас он отлично функционирует и его достаточно.